

SISTEM RESERVASI RESTORAN

Guna Memenuhi Tugas Mata Kuliah Algoritma dan Struktur Data

Dosen Pengampu:

Taufik Ridwan, M.T



Disusun Oleh: Kelompok 3

Anisa Putri Amelia	2510631250004
Line Budiarti	2510631250007
Muhammad Aji Nur Majid	2510631250037
Fathu Zaky Nugraha	2510631250061

PRODI SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS SINGAPERBANGSA KARAWANG

2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Tuhan Yang Maha Esa atas selesainya proyek tugas akhir mata kuliah Algoritma dan Struktur Data ini. Proyek "Sistem Reservasi Restoran" ini disusun untuk mengimplementasikan konsep struktur data *Array of Objects* serta algoritma *Searching* dan *Sorting* dalam skenario dunia nyata.

Kami mengucapkan terima kasih kepada Bapak Taufik Ridwan selaku dosen pengampu yang telah memberikan bimbingan, serta rekan-rekan Kelompok 3 yang telah berkontribusi dalam penyelesaian laporan ini. Kami menyadari laporan ini masih jauh dari sempurna, namun kami berharap proyek ini dapat memberikan gambaran fungsional mengenai manajemen data yang efisien.

Karawang, Mei 2026

Kelompok 3

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan bisnis restoran yang semakin pesat menuntut peningkatan kualitas pelayanan, terutama dalam pengelolaan reservasi meja. Sistem reservasi manual masih sering menimbulkan berbagai kendala, seperti kesalahan pencatatan, double booking, dan kesulitan memantau kapasitas ruangan secara real-time. Kondisi tersebut dapat menghambat operasional restoran dan menurunkan kepuasan pelanggan. Oleh karena itu, diperlukan sistem berbasis teknologi yang mampu mengelola data reservasi secara lebih efektif dan terstruktur.

Melalui penggunaan bahasa pemrograman Java dengan konsep pemrograman berorientasi objek, sistem reservasi dapat membantu proses pencatatan data pelanggan, area reservasi, jadwal kunjungan, serta status pemesanan secara terpusat menggunakan struktur data array of object. Dengan demikian, makalah ini membahas perancangan Sistem Reservasi Restoran Berbasis Java sebagai solusi pengelolaan reservasi yang lebih sistematis dan efisien.

B. Tujuan

1. Mendigitalisasi pencatatan reservasi restoran untuk menggantikan metode manual yang tidak efisien.
2. Menerapkan algoritma *sorting* dan *searching* untuk mempercepat pengelolaan dan pencarian data pelanggan.
3. Meningkatkan efisiensi operasional manajemen restoran dalam mengatur kapasitas area meja.

BAB II

ANALISIS KEBUTUHAN

A. Analisis Kebutuhan Fitur

1. Fitur Tambah Reservasi

Fitur ini digunakan untuk menambahkan data reservasi pelanggan baru ke dalam sistem.

Input yang digunakan:

- Nama pelanggan
- Kategori meja
- Tanggal reservasi
- Waktu reservasi
- Jumlah orang

Sistem secara otomatis memberikan ID pada setiap data reservasi baru.

2. Fitur Tampilkan Reservasi

Fitur ini digunakan untuk menampilkan seluruh data reservasi yang tersimpan dalam array.

Data yang ditampilkan:

- ID
- Nama
- Kategori
- Tanggal
- Waktu
- Jumlah orang
- Status

3. Fitur Edit Reservasi

Fitur edit digunakan untuk mengubah data reservasi berdasarkan ID pelanggan.

Data yang dapat diubah:

- Nama
- Kategori
- Tanggal
- Waktu
- Jumlah orang

4. Fitur Hapus Reservasi

Dalam fitur ini kami menggunakan soft delete dengan cara kami yaitu dengan mengganti status yang dari awalnya 'Aktif' menjadi 'Dihapus', serta kami juga menggunakan fitur 2-Step Verify (konfirmasi **y/n** dan pengetikan **SETUJU**) berguna agar data tidak semerta merta dihapus dengan mudah, dan juga kami menambahkan **3 sub-menu** yaitu mau dihapus secara **individu** (berdasarkan ID) atau dihapus secara **massal** (berdasarkan Nama) dan juga **reset sistem data**.

5. Fitur Searching

Program memiliki beberapa metode pencarian:

- Linear Search = berdasarkan nama
- Binary Search = berdasarkan ID
- Pencarian kategori = kategori, tanggal, waktu, dan jumlah orang

6. Fitur Sorting

Program memiliki beberapa metode pengurutan:

- Bubble Sort = berdasarkan ID dan jumlah orang
- Selection Sort = berdasarkan nama pelanggan

B. Analisis Struktur Data

1. Struktur Reservasi

Class `StrukturReservasi` digunakan untuk menyimpan data reservasi pelanggan.

Atribut	Tipe Data	Fungsi
ID	Int	Identitas Reservasi
Nama	String	Nama Pelanggan
Kategori	String	Jenis Reservasi
tanggal	String	Tanggal Reservasi
Waktu	String	Waktu Reservasi
Jumlah Orang	Int	Jumlah Pelanggan
Status	String	Status Reservasi

2. Array Object

Program menggunakan *Array of Objects* bernama `DataR` dengan alokasi ukuran statis sebesar 225 elemen (`MAX_DATA = 225`). Pembatasan kapasitas ini sengaja dilakukan demi menghemat penggunaan *resource* komputer agar tetap efisien, serta diestimasikan sudah sangat cukup untuk menampung seluruh data transaksi reservasi restoran selama 6 bulan ke depan.

BAB III

IMPLEMENTASI

A. Implementasi Kode dan Penjelasannya

Program Sistem Reservasi Restoran ini merupakan aplikasi berbasis bahasa pemrograman Java yang dirancang khusus untuk mengelola data reservasi pelanggan secara terstruktur. Program ini menerapkan konsep pemrograman berorientasi objek (*Object-Oriented Programming / OOP*) seperti *class* dan *object*, yang dikombinasikan dengan struktur data *array*, fungsi/*method*, algoritma pencarian (*searching*), pengurutan (*sorting*), serta manipulasi berkas (*file I/O*).

Seluruh implementasi logika, fungsi, dan antarmuka berbasis teks (*Command Line Interface*) dari sistem ini diintegrasikan secara utuh ke dalam satu file *source code* utama bernama `SistemReservasi_V1.java`.

B. Struktur Data

Program ini memanfaatkan struktur data linear berupa *Array of Objects* sebagai media penyimpanan utama di dalam memori komputer (*RAM*). *Array* digunakan untuk menampung kumpulan seluruh rekaman data secara sekuensial, sedangkan *object* digunakan untuk merepresentasikan entitas tunggal dari data reservasi pelanggan.

- **class StrukturReservasi**

Bertindak sebagai model data (*blueprint*) yang merepresentasikan atribut-atribut informasi dari setiap pelanggan yang melakukan reservasi. Atribut yang dideklarasikan di dalam kelas ini meliputi:

1. `ID (int)`: Kode unik identitas reservasi.
2. `nama (String)`: Nama lengkap pelanggan.
3. `kategori (String)`: Jenis kategori layanan/meja.
4. `tanggal (String)`: Tanggal pelaksanaan reservasi.
5. `waktu (String)`: Jam kehadiran pelanggan.
6. `jumlahOrang (int)`: Jumlah kuantitas tamu.
7. `Status (String)`: Status aktif atau pembatalan reservasi.

- **StrukturReservasi[] DataR**

Seluruh *object* reservasi yang dibuat akan disimpan ke dalam sebuah variabel *array static* bernama **DataR** dengan batas alokasi indeks maksimal sebesar **225** data (**MAX_DATA**). Struktur *array* konvensional sengaja dipilih dalam proyek ini karena memiliki performa akses indeks yang konstan $\mathcal{O}(1)$ serta mempermudah implementasi algoritma *searching* dan *sorting* secara mandiri tanpa ketergantungan pada *library* koleksi bawaan Java.

C. Menu Utama

Arsitektur antarmuka program dirancang terpusat pada menu utama yang dikendalikan di dalam struktur perulangan **do-while** dan percabangan **switch-case**. Mekanisme ini berfungsi untuk mengatur navigasi fitur program secara interaktif berdasarkan input angka yang dimasukkan oleh pengguna.

Ketika pengguna memilih opsi menu tertentu, program secara dinamis akan memicu fungsi (*method static*) spesifik untuk mengeksekusi operasi tersebut. Pemetaan struktur menu utama didefinisikan sebagai berikut:

- Menu 1 → tambah reservasi
- Menu 2 → tampilkan data
- Menu 3 → edit data
- Menu 4 → hapus data
- Menu 5 → pencarian data
- Menu 6 → Urutkan Reservasi
- Menu 7 → Tampilkan Statistik
- Menu 8 → Simpan/Load Data
- Menu 0 → Keluar

BAB IV

ANALISIS KOMPLEKSITAS

A. Tabel Analisis Kompleksitas

Kategori Fitur	Nama Fungsi di Kode	Best Case	Average Case	Worst Case	Space Complexity
Searching Features	<code>linearSearchNama()</code>	$O(N)$	$O(N)$	$O(N)$	$O(1)$
	<code>binarySearchId()</code>	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$
	<code>cariKategori()</code>	$O(N)$	$O(N)$	$O(N)$	$O(1)$
Sorting Features	<code>bubbleSortID()</code>	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
	<code>selectionsortNama()</code>	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
	<code>SortByJumlah()</code>	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$

Tabel 4.1

B. Penjelasan Kompleksitas

- Algoritma Pencarian (Search):

Linear Search (`linearSearchNama()` & `cariKategori()`). Memiliki kompleksitas $O(N)$ pada semua kasus termasuk worst case. Jika data reservasi yang dicari berada di elemen paling akhir atau tidak ada, program harus mengecek setiap elemen array satu per satu dari awal sampai batas total data aktif.

- Binary Search (`binarySearchId()`):

Jauh lebih efisien dengan kompleksitas $O(\log N)$ pada worst case. Karena data wajib diurutkan terlebih dahulu, algoritma ini membagi rentang pencarian menjadi dua secara terus-menerus sehingga proses penemuan ID reservasi berjalan sangat cepat.

- Algoritma Pengurutan (Sort):

Bubble Sort (`bubbleSortID()` & `SortByJumlah()`) & Selection Sort (`selectionsortNama()`). Fungsi-fungsi ini memiliki kompleksitas waktu $O(N^2)$ pada best, average, maupun worst case. Hal ini disebabkan logika code nya menggunakan dua perulangan bersarang (nested loops) yang bekerja secara penuh untuk membandingkan dan menukar elemen objek reservasi tanpa adanya fungsi pemberhentian awal (flag).

- Kompleksitas Ruang (Space Complexity):

Seluruh fitur pencarian dan pengurutan di atas memiliki kompleksitas ruang sebesar $O(1)$. Hal ini dikarenakan operasi dilakukan secara in-place. Artinya, program memodifikasi dan menukar posisi objek data secara langsung di dalam memori Array of Objects yang sudah dialokasikan sejak awal tanpa memerlukan struktur data atau memori tambahan.

BAB V

PENGUJIAN

A. Pengujian CRUD

- Input Reservasi Baru

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [1] **Tambah Reservasi Baru**
2. Setelah itu Masukkan Nama, kategori, Tanggal, Waktu dan jumlah orang yang akan di reservasi.
3. Maka semua data akan ter-display seperti pada gambar di bawah.
4. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama.**

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 1
-----

--- TAMBAH RESERVASI BARU ---
[1] lanjutkan menambah data reservasi baru
[2] Kembali ke Menu Utama
Pilih opsi: [1-2] 1
Masukkan nama: Aji
Masukkan kategori (Reguler/Outdoor): reguler
Masukkan tanggal (DD-MM-YYYY): 14-06-2026
Masukkan waktu (HH:MM): 14:00
Masukkan jumlah orang: 4
---Reservasi berhasil ditambahkan.---
```

Gambar 5.1

Gambar 5.1 menunjukkan sistem berhasil membuat objek **StrukturReservasi** baru, meningkatkan counter totalData, dan memberikan ID otomatis secara sekuensial.

- Menampilkan Semua Data

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [2] **Tampilkan Semua Reservasi**.
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi [2] **Tampilkan Semua detail reservasi (tanpa Terkecuali)**.
3. Maka semua data akan ter-display seperti pada gambar di bawah.
4. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama**.

SISTEM RESERVASI RESTORAN					
[1] Tambah Reservasi Baru [2] Tampilkan Semua Reservasi [3] Edit Reservasi [4] Hapus Reservasi [5] Cari Reservasi [6] Urutkan Reservasi [7] Tampilkan Statistik [8] Simpan/Load Data ke File [0] Keluar					
[?] Pilih menu (0-8): 2					
=====					
=== OPSI TAMPILKAN DATA ===					
[1] Tampilkan Ringkasan Total Data (Berupa Angka)					
[2] Tampilkan Semua Detail Reservasi (Tanpa Terkecuali)					
[3] Kembali ke Menu Utama					
Pilih opsi tampilan: 2					
===== DAFTAR SEMUA DETAIL RESERVASI =====					
1	Badi	Outdoor	4	Aktif	
2	Audi	Reguler	2	Aktif	
3	Siti	Outdoor	6	Aktif	
4	Rina	Reguler	3	Aktif	
5	Dewi	Outdoor	5	Aktif	
6	Andi	Reguler	4	Aktif	
7	Lina	Outdoor	2	Aktif	
8	Rudi	Reguler	3	Aktif	
9	Maya	Outdoor	6	Aktif	
10	Fajar	Reguler	4	Aktif	
11	Aji	Outdoor	2	Aktif	
12	Sari	Reguler	6	Aktif	
13	Rina	Outdoor	3	Aktif	
14	Dewi	Reguler	5	Aktif	
15	Andi	Outdoor	4	Aktif	
16	Lina	Reguler	2	Aktif	
17	Rudi	Outdoor	3	Aktif	
18	Maya	Reguler	6	Aktif	
19	Fajar	Outdoor	4	Aktif	
20	Aji	Reguler	2	Aktif	
21	Sari	Outdoor	6	Aktif	
22	Rina	Reguler	10	Aktif	
23	Andi	Reguler	4	Aktif	
24	Lina	Outdoor	2	Aktif	
25	Rudi	Reguler	3	Aktif	
26	Maya	Outdoor	6	Aktif	
27	Dewi	Outdoor	5	Aktif	
28	Line	Outdoor	3	Aktif	
29	Nisa	Reguler	3	Aktif	
30	Zaky	Outdoor	3	Aktif	
31	Aji	reguler	4	Aktif	

Gambar 5.2

Gambar 5.2 menunjukkan sistem berhasil melakukan iterasi dan mencetak seluruh data reservasi yang aktif ke layar secara rapi.

- Edit Data Berdasarkan ID

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [3] Edit Reservasi.
2. Sistem meminta kita untuk memasukan ID yang ingin diedit, dan setelah itu akan terdisplay Nama, kategori, Tanggal, Waktu dan jumlah orang sesuai ID yang kita mau Edit
3. Kita dimintai mengisi Nama, kategori, Tanggal, Waktu dan jumlah orang dengan yang baru, dan hasilnya ada di gambar di bawah.
4. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] Opsi tidak valid. Kembali ke menu utama.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 3
-----
Masukkan ID reservasi yang ingin diedit: 1
Data ditemukan
Nama: Badi
Kategori: Outdoor
Tanggal: 15-05-2026
Waktu: 18:00
Jumlah Orang: 4
Apakah Anda yakin ingin mengubah data ini [y/n]: y
Masukkan nama baru           : Hendra
Masukkan kategori baru (Reguler/Outdoor): Reguler
Masukkan tanggal baru (DD-MM-YYYY) : 15-06-2026
Masukkan waktu baru (HH:MM)       : 11:00
Masukkan jumlah orang baru       : 4
Reservasi berhasil diubah.
```

Gambar 5.3

Gambar 5.3 membuktikan bahwa fungsi berhasil mencari indeks ID target dan menimpa nilai atribut lama dengan input yang baru.

- Delete Data (2-Step Verify)

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [4] **Hapus Reservasi**.
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi [1] **Hapus Data Individual (Berdasarkan ID)**.
3. Pengguna memasukkan ID target, yaitu 1.
4. Sistem melakukan *2-step verification*: konfirmasi pertama dengan memasukkan karakter **y**, dan konfirmasi finalisasi keamanan dengan mengetikkan kata kunci **SETUJU**.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 4
-----
=== [4] MENU HAPUS DATA ===
[1] Hapus Data Individual (Berdasarkan ID)
[2] Hapus Data Massal (Berdasarkan Nama)
[3] Hapus SEMUA Data (Reset Sistem)
[4] Kembali ke Menu Utama
Pilih opsi penghapusan [1-4]: 1

Masukkan ID reservasi yang ingin dihapus: 1

--- DATA YANG AKAN DIHAPUS ---
ID      : 1
Nama    : Hendra
Kategori: Reguler
-----
Apakah Anda yakin ingin menghapus data ini? (y/n): y
Ketik 'SETUJU' untuk memfinalisasi: SETUJU
Delete berhasil. Reservasi ID 1 telah dinonaktifkan.
```

Gambar 5.4

Berdasarkan **Gambar 5.4**, fitur penghapusan data telah berjalan sesuai spesifikasi sistem. Sistem berhasil menemukan data dengan ID 1 atas nama "Hendra" dan menampilkannya terlebih dahulu sebelum dieksekusi demi menghindari salah hapus. Fitur keamanan *2-step verification* terbukti berfungsi memvalidasi niat pengguna. Setelah proses konfirmasi selesai, sistem memicu mekanisme *soft delete* dengan mengubah nilai atribut status objek menjadi non-aktif, dibuktikan dengan keluaran pesan log **"Delete berhasil. Reservasi ID 1 telah dinonaktifkan."**

B. Pengujian Fitur Algoritma Searching

- Cari Berdasarkan Nama (Linear Search)

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [5] **Cari Reservasi**.
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metode pencarian [1] **Cari berdasarkan Nama**.
3. Pengguna memasukan nama yang ingin dicari **Masukkan Nama yang dicari: Aji**
4. Jika nama ditemukan di sistem maka akan keluar hasilnya, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama**

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 5
-----
=== MENU CARI RESERVASI ===
[1] Cari berdasarkan Nama
[2] Cari berdasarkan ID
[3] Cari berdasarkan Kategori Lainnya (Tanggal/Waktu/Jumlah Orang)
[4] Kembali ke Menu Utama
Pilih metode pencarian: [1-4] 1
Masukkan Nama yang dicari: Aji

----- Hasil Pencarian Nama -----
| 11 | Aji | Outdoor | 2 | Aktif |
| 20 | Aji | Reguler | 2 | Aktif |
```

Gambar 5.5

Gambar 5.5 membuktikan fungsi `linearSearchNama()` berhasil memindai seluruh *array* dan menampilkan semua data reservasi yang mengandung substring nama yang dicari (fitur pencocokan `.contains()`).

- **Cari Berdasarkan ID (*Binary Search*)**

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [5] **Cari Reservasi**.
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metode pencarian [2] **Cari berdasarkan ID**.
3. Pengguna memasukkan ID yang ingin dicari **Masukkan ID yang dicari: 2**
4. Jika ID ditemukan di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama**. dan apabila ID ditemukan namun sudah terhapus maka **Data dengan ID 1 ditemukan, tetapi statusnya sudah 'Dihapus'**.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 5
-----
=== MENU CARİ RESERVASI ===
[1] Cari berdasarkan Nama
[2] Cari berdasarkan ID
[3] Cari berdasarkan Kategori Lainnya (Tanggal/Waktu/Jumlah Orang)
[4] Kembali ke Menu Utama
Pilih metode pencarian: [1-4] 2
Masukkan ID yang dicari: 2
| 2 | Audi | Reguler | 2 | Aktif |
```

Gambar 5.6

Gambar 5.6 menunjukkan fungsi `binarySearchId()` berhasil berjalan. Sistem melakukan pengurutan ID terlebih dahulu sebelum membagi ruang pencarian secara biner hingga ID ditemukan dengan efisien.

- Cari Berdasarkan Kategori

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [5] **Cari Reservasi**.
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metode pencarian [3] **Cari berdasarkan Kategori**.
3. Pengguna memasukkan Kategori (Jumlah Orang) yang ingin dicari **Masukkan Kategori yang dicari (Tanggal/Waktu/Jumlah Orang): 5**
4. Jika Kategori ditemukan di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama**.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 5
-----
=== MENU CARI RESERVASI ===
[1] Cari berdasarkan Nama
[2] Cari berdasarkan ID
[3] Cari berdasarkan Kategori lainnya (Tanggal/Waktu/Jumlah Orang)
[4] Kembali ke Menu Utama
Pilih metode pencarian: [1-4] 3
Masukkan Kategori yang dicari (Kategori/Tanggal/Waktu/Jumlah Orang): 5
----- Hasil Pencarian Kategori -----
| 5 | Dewi | Outdoor | 5 | Aktif |
| 14 | Dewi | Regular | 5 | Aktif |
| 27 | Dewi | Outdoor | 5 | Aktif |
```

Gambar 5.7

Gambar 5.7 menunjukkan fungsi `cariKategori()` berhasil menyaring (*filtering*) dan memunculkan semua kelompok reservasi yang sesuai dengan kategori tersebut mulai dari tanggal, waktu dan jumlah orang hanya dengan mengetik salah satu antara tanggal, waktu dan jumlah orang tanpa memilih terlebih dahulu.

C. Pengujian Fitur Algoritma Sorting

- Urutkan Berdasarkan ID (*Ascending - Bubble Sort*)

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [6] **Urutkan Reservasi**
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metodeurut [1] **Mengurutkan data berdasarkan ID**.
3. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama**.
4. Untuk melihat hasil dari pengurutannya dapat dilihat dengan memilih opsi menu 2/ kembali ke menu prosedur menu tampilkan semua reservasi.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 6
-----
Pilih metode pengurutan:
[1] Mengurutkan data berdasarkan ID
[2] Mengurutkan data berdasarkan Nama
[3] Mengurutkan Berdasarkan Jumlah Orang Terbanyak
[4] Kembali ke Menu Utama
Pilihan: [1-4] 1
--- Data berhasil diurutkan. Silakan cek menu Tampilkan Semua Reservasi. ---
```

Gambar 5.8

Gambar 5.8 menunjukkan fungsi `bubbleSortID()` berhasil memindahkan dan menyusun elemen objek secara menaik (*ascending*) berdasarkan nilai ID.

- **Urutkan Berdasarkan Nama (*Alphabetical - Selection Sort*)**

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [6] **Urutkan Reservasi**
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metodeurut [2] **Mengurutkan data berdasarkan Nama**.
3. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama**.
4. Untuk melihat hasil dari pengurutanya dapat dilihat dengan memilih opsi menu 2/ kembali ke menu prosedur menu tampilkan semua reservasi.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 6
-----
Pilih metode pengurutan:
[1] Mengurutkan data berdasarkan ID
[2] Mengurutkan data berdasarkan Nama
[3] Mengurutkan Berdasarkan Jumlah Orang Terbanyak
[4] Kembali ke Menu Utama
Pilihan: [1-4] 2
--- Data berhasil diurutkan. Silakan cek menu Tampilkan Semua Reservasi. ---
```

Gambar 5.9

Gambar 5.9 menunjukkan fungsi `selectionsortNama()` berhasil mengurutkan nama pelanggan dari abjad terdepan hingga paling akhir menggunakan metode perbandingan nilai *String*.

- **Urutkan Berdasarkan Jumlah Orang Terbanyak (*Descending - Bubble Sort*)**

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [6] **Urutkan Reservasi**
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metodeurut [3] **Mengurutkan data berdasarkan Jumlah Orang Terbanyak**
3. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama.**
4. Untuk melihat hasil dari pengurutanya dapat dilihat dengan memilih opsi menu 2/ kembali ke menu prosedur menu tampilkan semua reservasi.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 6
-----
Pilih metode pengurutan:
[1] Mengurutkan data berdasarkan ID
[2] Mengurutkan data berdasarkan Nama
[3] Mengurutkan Berdasarkan Jumlah Orang Terbanyak
[4] Kembali ke Menu Utama
Pilihan: [1-4] 3
--- Data berhasil diurutkan. Silakan cek menu Tampilkan Semua Reservasi. ---
```

Gambar 5.10

Gambar 5.10 menunjukkan fungsi **SortByJumlah()** berhasil melakukan pengurutan menurun (*descending*) bersandarkan jumlah orang dalam reservasi.

D. Pengujian Fitur Tambahan

- Tampilkan Statistik Data

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [7] **Tampilkan Statistik**
2. Maka hasilnya akan terlihat langsung di dalam terminal.

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 7
-----

|-----|
| STATISTIK RESERVASI RESTORAN |
|-----|
| [ SUMMARY ] |
| Total Semua Record      : 30 data |
| Reservasi Sesi Aktif     : 29 reservasi |
| Reservasi Dibatalkan     : 1 reservasi |
| Total Tamu Hadir        : 117 orang |
|-----|
| [ TREN KATEGORI (Sesi Aktif) ] |
| Kategori Reguler        : 14 pesanan |
| Kategori Outdoor        : 15 pesanan |
|-----|
| -> Terfavorit : Outdoor (15 pesanan) |
|-----|
| [ REKAP KAPASITAS PER TANGGAL ] |
| Tanggal 15-05-2026      : 2 orang |
| Tanggal 17-05-2026      : 14 orang |
| Tanggal 20-05-2026      : 8 orang |
| Tanggal 22-05-2026      : 11 orang |
| Tanggal 25-05-2026      : 11 orang |
| Tanggal 28-05-2026      : 9 orang |
| Tanggal 30-05-2026      : 5 orang |
| Tanggal 01-06-2026      : 12 orang |
| Tanggal 04-06-2026      : 20 orang |
| Tanggal 08-06-2026      : 10 orang |
| Tanggal 10-06-2026      : 15 orang |
|-----|
```

Gambar 5.11

Gambar 5.11 menampilkan kalkulasi total data, data yang aktif, serta pemetaan tren reservasi, dan rekap kapasitas Per tanggal, berguna untuk mengetahui berapa jumlah orang yang ada di dalam tanggal tertera yang berhasil diproses oleh sistem.

- **Simpan dan Load Data ke File Teks**

Prosedur:

1. Pengguna masuk ke menu utama dan memilih opsi [8] **Simpan/Load Data ke File**
2. Sistem menampilkan sub-menu, kemudian pengguna memilih opsi metodeurut
 - [1] **Simpan/Backup Data ke File**
 - [2] **Muat/Restore Data dari File**
 - [3] **Kembali ke Menu Utama**
3. Jika benar dalam memilih Sub-menu di sistem maka akan keluar hasilnya atau, jika tidak maka sistem akan mengalihkan pengguna ke menu utama [X] **Opsi tidak valid. Kembali ke menu utama.**

```
SISTEM RESERVASI RESTORAN

[1] Tambah Reservasi Baru
[2] Tampilkan Semua Reservasi
[3] Edit Reservasi
[4] Hapus Reservasi
[5] Cari Reservasi
[6] Urutkan Reservasi
[7] Tampilkan Statistik
[8] Simpan/Load Data ke File
[0] Keluar

[?] Pilih menu (0-8): 8
-----

=== KELOLA FILE DATA (I/O) ===
[1] Simpan/Backup Data ke File
[2] Muat/Restore Data dari File
[3] Kembali ke Menu Utama
Pilih opsi: 1
SUKSES: 30 data berhasil disimpan ke dalam file data_reservasi.txt
```

Gambar 5.12

Gambar 5.12 membuktikan manajemen file eksternal berjalan sukses, program mampu membaca (*parsing*) data lama kembali ke dalam *Array of Objects* sehingga data bersifat permanen (*persistent*).

BAB VI

KESIMPULAN

A. Kesimpulan

Sistem Reservasi Restoran Berbasis Java ini berhasil mendigitalisasi proses pencatatan manual menjadi jauh lebih terstruktur, terpusat, dan aman. Melalui kombinasi implementasi algoritma pencarian (*Searching*) dan pengurutan (*Sorting*), sistem mampu mempercepat operasional pengelolaan data pelanggan secara efisien. Sistem ini juga telah dilengkapi dengan fitur keamanan *Soft Delete* dengan mekanisme *2-Step Verification*, fitur manajemen *file I/O* untuk menyimpan data permanen, pendataan statistik yang dinamis, serta mekanisme *Overbooking Prevention* (pencegahan kelebihan kapasitas maksimal 30 orang pada hari yang sama).

Meskipun demikian, sistem masih memiliki keterbatasan pada kapasitas penyimpanan memori utama yang bersifat statis (terbatas pada alokasi maksimal 225 data), sehingga sistem ini masih membutuhkan tahap pengembangan berskala besar di masa mendatang.

B. Analisis Sistem

Berdasarkan hasil perancangan dan pengujian, berikut adalah analisis kelebihan dan kekurangan dari Sistem Reservasi Restoran yang telah dibangun:

Kelebihan:

1. Menggunakan struktur data yang sederhana, mudah dipahami, namun tetap efisien $O(1)$ secara alokasi memori (*Space Complexity*).
2. Memiliki fitur operasi CRUD (*Create, Read, Update, Delete*) yang lengkap dan fungsional.
3. Dilengkapi sistem keamanan validasi *2-step verification* untuk menghindari *human-error* saat menghapus data.
4. Mendukung pemantauan visualisasi tren statistik reservasi yang dinamis.
5. Memiliki fitur *Overbooking Prevention* otomatis untuk menolak reservasi jika kapasitas melampaui batas harian.

Kekurangan:

1. Masih bergantung pada penggunaan *Array of Objects* yang bersifat statis, sehingga kapasitas penyimpanan tidak bisa meluas secara fleksibel.
2. File penyimpanan eksternal (`data_reservasi.txt`) belum sepenuhnya aman karena basis datanya belum terenkripsi.
3. Arsitektur sistem belum mendukung akses oleh banyak pengguna (*multi-user*) di waktu yang bersamaan.
4. Efisiensi algoritma pengurutan masih tergolong rendah dengan *Time Complexity* pada tingkat $O(N^2)$.
5. Belum terintegrasi secara langsung dengan sistem *Database* sungguhan.

C. Lesson Learned (Pembelajaran)

Selama proses pengerjaan proyek Sistem Reservasi Restoran ini, terdapat beberapa pembelajaran berharga yang berhasil didapatkan oleh tim pengembang, di antaranya:

1. Mendapatkan pemahaman dan pengalaman baru terkait penulisan sintaks-sintaks dalam bahasa pemrograman Java.
2. Mempelajari bagaimana cara menjembatani dan mengaplikasikan langsung teori-teori algoritma (seperti *Linear/Binary Search* dan *Bubble/Selection Sort*) ke dalam skenario kasus dunia nyata.
3. Menyadari bahwa perancangan logika manajemen data yang aman, efisien, dan terstruktur merupakan fondasi yang sangat krusial dalam pengembangan sebuah perangkat lunak.

D. Saran Pengembangan

Untuk menutupi kekurangan pada sistem, disarankan pada iterasi pengembangan selanjutnya agar program dapat diintegrasikan dengan *Relational Database Management System (RDBMS)* seperti MySQL guna menggantikan *array statis*. Selain itu, sangat disarankan untuk menerapkan algoritma *sorting* tingkat lanjut (seperti *Quick Sort* atau

Merge Sort) dan mengimplementasikan sistem enkripsi file (*Cryptography*) agar keamanan privasi data pelanggan lebih terjamin.